

초급 백엔드 스터디

4주차 - 레포지토리 계층

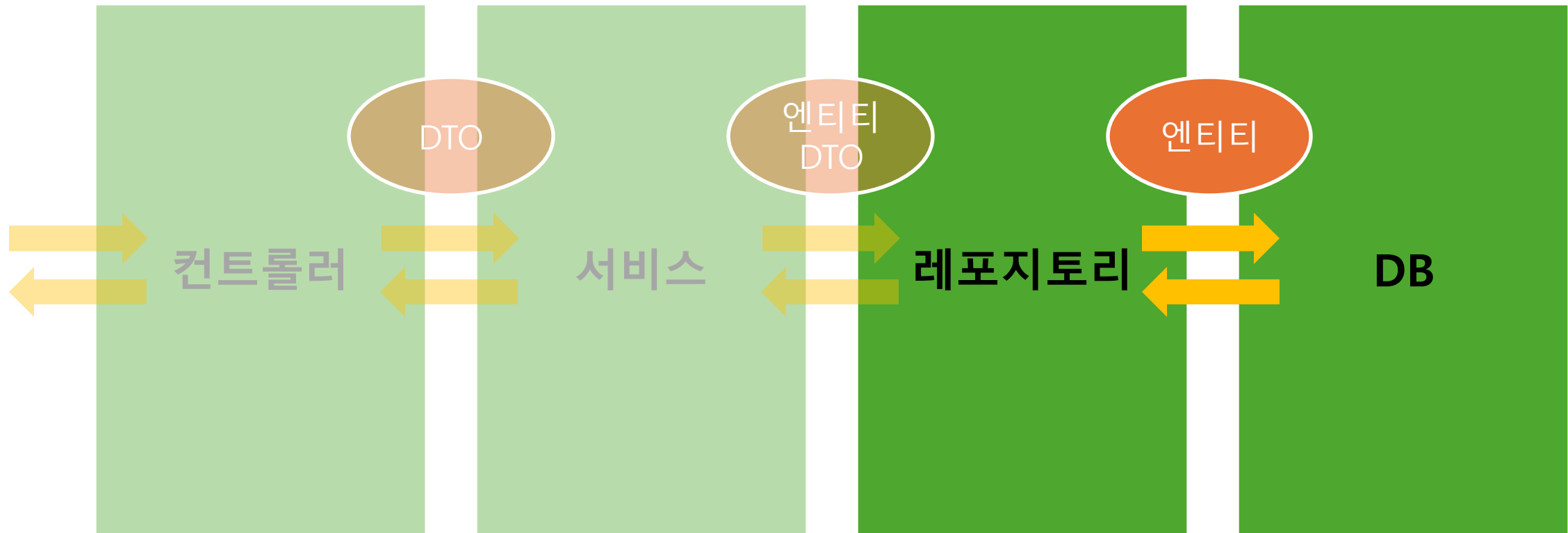
지난 주에는...

- ERD - DB 구조 설계
- JPA - 자바 객체와 DB 매핑 도구
- 엔티티 - JPA가 DB와 소통하는 데이터 단위

이번 주에는...

- JPA를 사용하여 만든 테이블에
데이터를 넣고, 읽고, 수정하고, 삭제해보자

스프링 Layered Architecture



레포지토리 계층

- **DB**와 소통하며 데이터를 조작하는 계층
- 서비스 계층이 결정한 비즈니스 로직을 실제 DB에 적용한다.

레포지토리 계층

데이터 조작은 크게 4가지 기능을 위주로 구현하면 된다.
이를 통틀어 CRUD 라고 부른다.

- 생성 (Create)
- 조회 (Read)
- 수정 (Update)
- 삭제 (Delete)

레포지토리 계층

- CRUD 기능은 **JPA가 제공하는 기능을 이용하여** 구현한다.

엔티티 매니저

- JPA는 application.yml 정보를 통해 **Entity Manager**를 생성한다.
- **엔티티 매니저**는 우리 대신 DB와 직접 소통하는 객체

엔티티 매니저가 하는 일

- 새로 생성한 엔티티 객체를 DB에 추가
- DB에서 조회한 데이터로 엔티티 객체 만들기
- 엔티티 객체에 대한 수정, 삭제를 DB에 반영하기

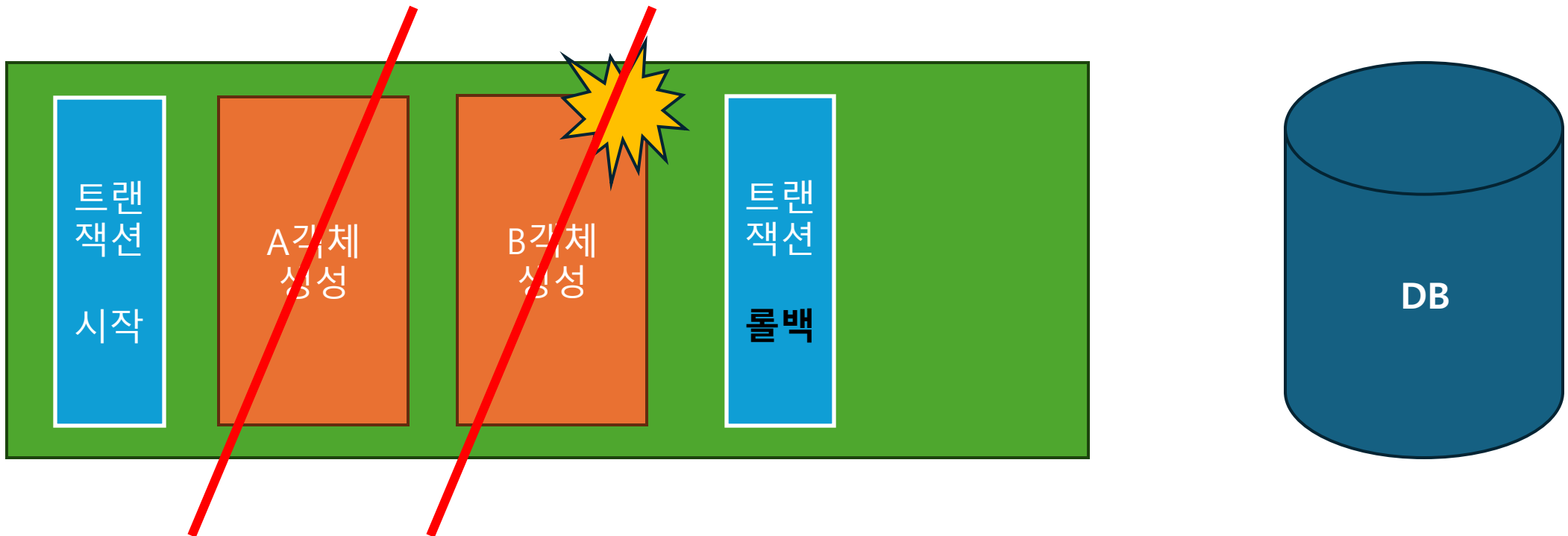
트랜잭션

- JPA는 DB와 유사하게 **트랜잭션 단위**로 동작한다.
- 트랜잭션이 끝나면 **모든 변경사항을 DB에 반영**한다.



트랜잭션

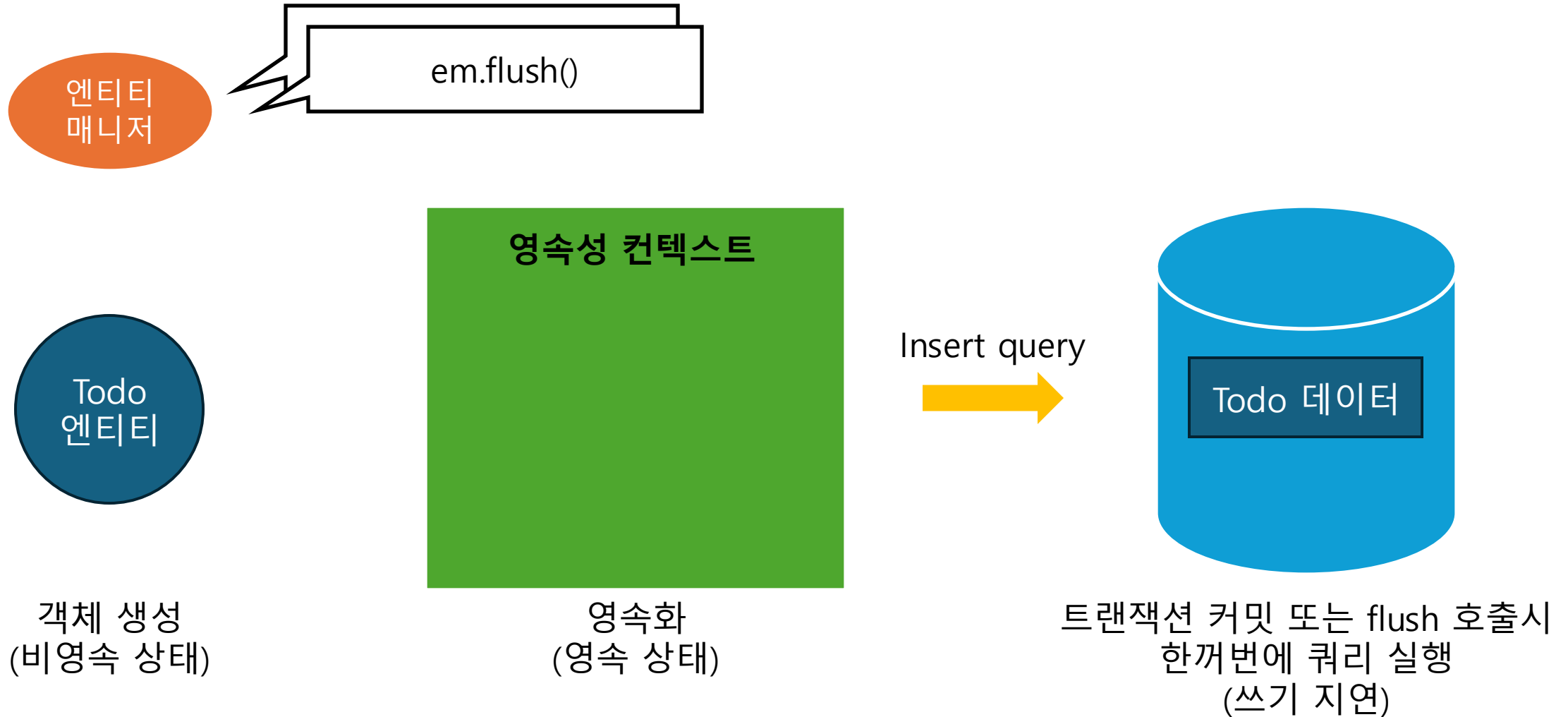
- 트랜잭션 중간에 에러가 발생하면
트랜잭션 범위 안의 모든 변경점을 되돌린다. (롤백)



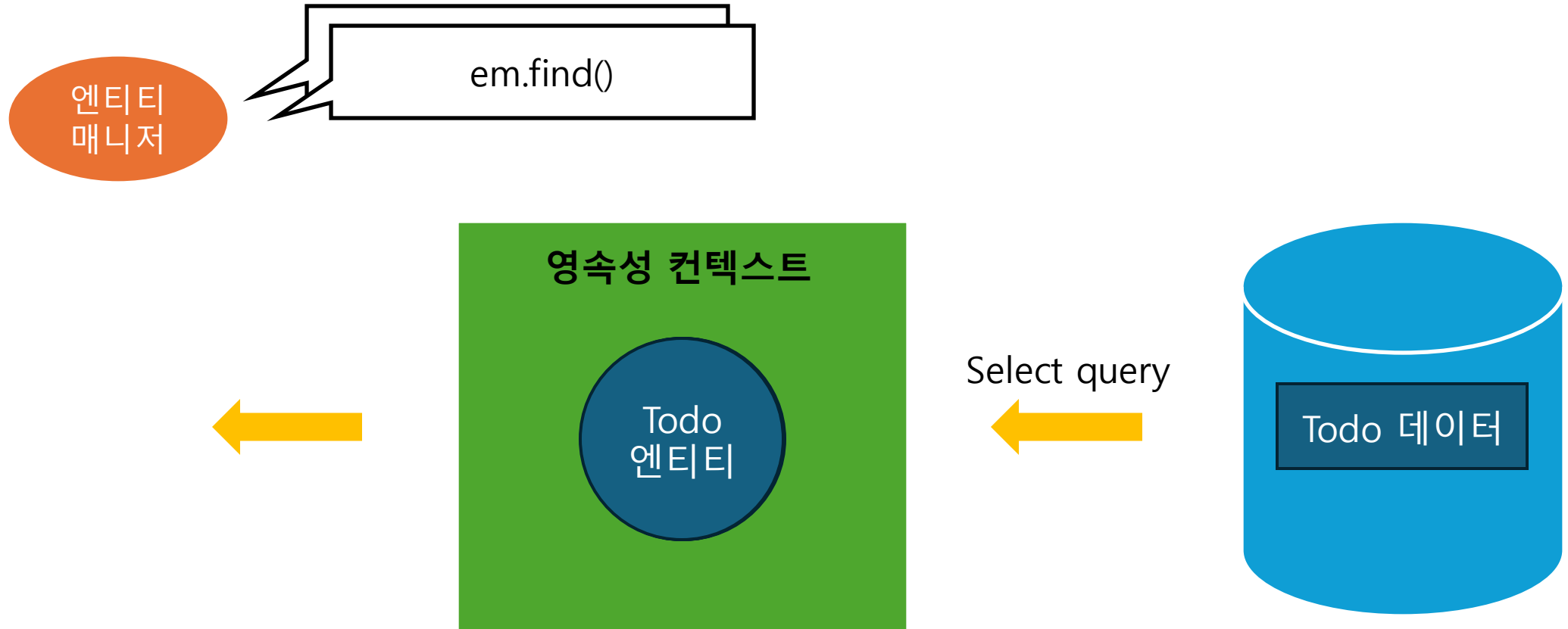
영속성 컨텍스트

- DB에서 조회한 엔티티를 캐싱하는 공간
- JPA가 DB에 반영할, 엔티티의 모든 변경 사항을 보관하는 공간
(일종의 버퍼)
- 엔티티에 대한 변경 사항을 영속성 컨텍스트에 저장해 두었다가, 트랜잭션을 커밋하면 저장된 모든 변경점이 DB에 반영되도록 영속성 컨텍스트를 기반으로 한번에 SQL을 생성한다.

엔티티 저장

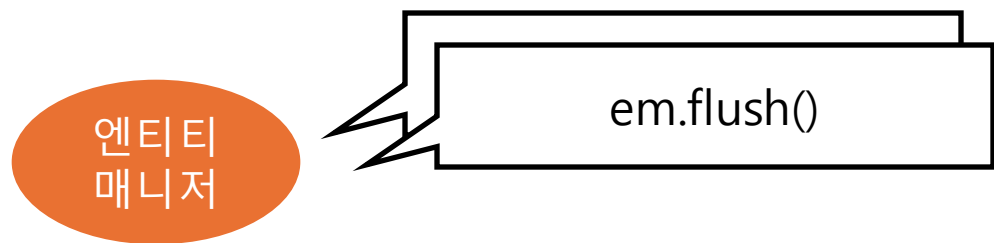


엔티티 조회



두번째 조회시에는 DB에 쿼리를 보내지 않고
영속성 컨텍스트에서 가져온다.
(1차 캐시, 동일 객체 보장)

엔티티 수정



Update query

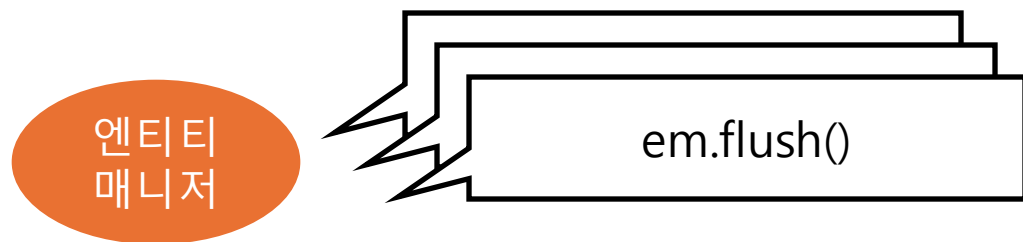
A yellow double-headed arrow pointing both left and right, indicating a bidirectional relationship between the persistence context and the database.



컨텍스트 안에 있는 객체를 수정 시,
수정된 정보와 최초 상태를 비교해서 SQL 생성
(변경 감지)

트랜잭션 커밋 또는 flush 호출시
생성된 쿼리들을 한번에 실행
(쓰기 지연)

엔티티 삭제



컨텍스트 안에 있는 객체를 삭제 시,
엔티티를 삭제 상태로 만든다.

Delete query

A yellow double-headed arrow pointing left and right, indicating a bidirectional relationship or flow.



트랜잭션 커밋 또는 flush 호출시
생성된 쿼리들을 한번에 실행
(쓰기 지연)

영속성 컨텍스트

정리하면...

- 엔티티 매니저는 변경 사항을 모았다가, **한번에 SQL을 생성한다.**
- 이때 모든 변경 사항은 **영속성 컨텍스트에 저장된다.**

영속성 컨텍스트

엔티티 매니저에게 데이터 변경을 요청하고 싶다면

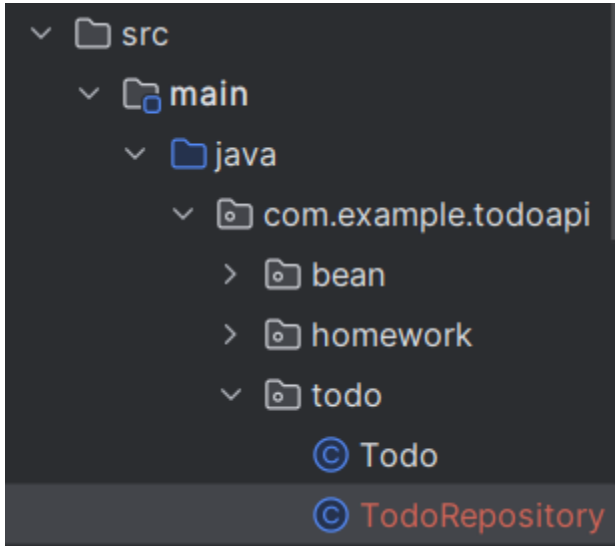
하나의 데이터를 나타내는 **'엔티티 객체'**를 **영속성 컨텍스트**에 올려두고
영속성 컨텍스트에 올려둔 **엔티티 객체를 변경**하면 된다.

레포지토리 계층

- 엔티티 매니저를 사용해서 레포지토리 계층을 구현해보자.

레포지토리 계층

- Todo 패키지에 TodoRepository 클래스 추가



레포지토리 계층

- 레포지토리는 빈으로 등록하여 사용한다.
- **@Repository** 어노테이션을 사용한다.
(내부적으로 @Component 어노테이션이 적용되어 컴포넌트 스캔 시 빈으로 등록된다.)

```
@Repository
public class TodoRepository {
}
```

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface Repository {

    | Alias for Component.value.
    |
    @AliasFor(annotation = Component.class)
    String value() default "";

}
```

레포지토리 계층

- 영속성 컨텍스트에 접근할 엔티티 매니저를 가져온다.

```
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

import org.springframework.stereotype.Repository;

@Repository
public class TodoRepository {

    no usages
    @PersistenceContext
    private EntityManager em;
}
```

할 일 생성

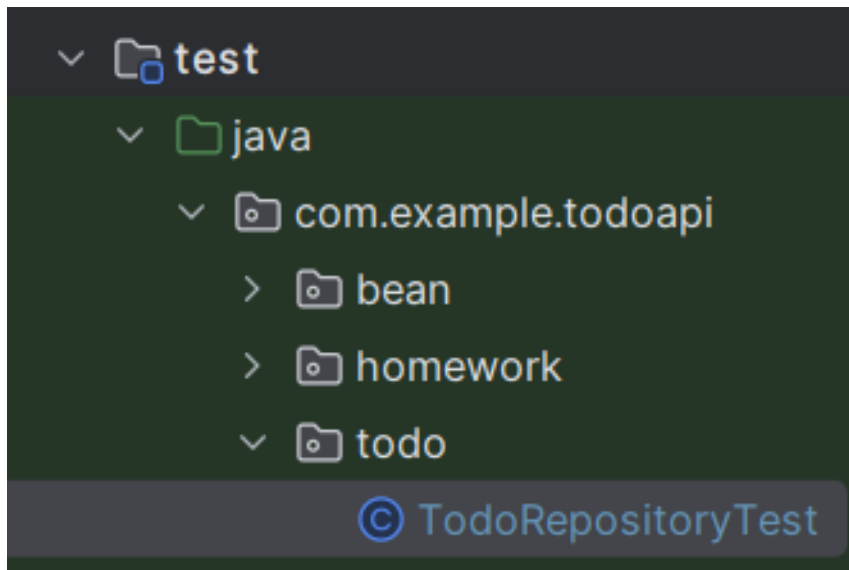
- **em.persist()**
- 영속성 컨텍스트에 엔티티 추가
트랜잭션 커밋시 DB에도 데이터를 추가한다.
(DB를 영속성 컨텍스트와 동기화)

```
2 usages
@PersistenceContext
private EntityManager em;

1 usage
public void save(Todo todo) {
    em.persist(todo);
}
```

레포지토리 테스트

- DB에 데이터가 잘 생성되는지 테스트해보자.
- test 패키지 아래에 todo 패키지를 생성하고, 테스트 클래스 생성



레포지토리 테스트

- 실제 DB와 통신하면서 데이터를 넣으려면 테스트할 때도 스프링 부트를 실행시켜야 한다.

```
no usages  👤 kwon *  
@SpringBootTest(webEnvironment = WebEnvironment.DEFINED_PORT)  
public class TodoRepositoryTest {  
  
}
```



테스트 용도의 가짜 웹 환경이 아닌
실제 웹 환경을 띄워서 테스트한다.
이때 웹 접속은 8080 포트로 접속
(관리자 콘솔에 접속해보기 위함)

레포지토리 테스트

- 테스트가 끝나면 어플리케이션이 종료되며 **DB가 사라짐**
- 따라서 어플리케이션이 종료되지 않도록 반복문 추가

```
@SpringBootTest(webEnvironment = WebEnvironment.DEFINED_PORT)
public class TodoRepositoryTest {

    @kwon
    @AfterAll
    static void test() {
        System.out.println("test finish");
        while(true) {

        }
    }
}
```

모든 테스트가 끝난 뒤
실행하는 메서드

레포지토리 테스트

- 테스트할 레포지토리 빈을 주입받는다.

```
@SpringBootTest(webEnvironment = WebEnvironment.DEFINED_PORT)
public class TodoRepositoryTest {

    @Autowired
    private TodoRepository todoRepository;

    @AfterAll
    static void test() {
        System.out.println("test finish");
        while(true) {
        }
    }
}
```

할 일 생성 테스트

- 아래와 같이 테스트를 작성한다.

```
@Test
@Transactional
@Rollback(value = false)
void todoCreateTest() {
    // given
    Todo todo = new Todo( content: "content", isChecked: false, user: null);

    // when
    todoRepository.save(todo);

    // then
    assertThat(todo.getId()).isNotNull();
}
```

엔티티 매니저를 사용할 때,
트랜잭션 안에서 사용해야 한다.
@Transactional 을 붙이면
메서드 범위에서
트랜잭션 시작, 커밋, 롤백을
알아서 처리해준다.

할 일 생성 테스트

- 아래와 같이 테스트를 작성한다.

```
@Test
@Transactional
@Rollback(value = false)
void todoCreateTest() {
    // given
    Todo todo = new Todo( content: "content", isChecked: false, user: null);

    // when
    todoRepository.save(todo);

    // then
    assertThat(todo.getId()).isNotNull();
}
```

테스트 환경에서는
하나의 테스트가 끝나면 자동으로
롤백해서 DB를 되돌린다.
DB에 데이터가 생성된 것을
확인하기 위해 롤백을 하지 않도록
설정한다.

할 일 생성 테스트

- 아래와 같이 테스트를 작성한다.

```
@Test
@Transactional
@Rollback(value = false)
void todoCreateTest() {
    // given
    Todo todo = new Todo( content: "content", isChecked: false, user: null);

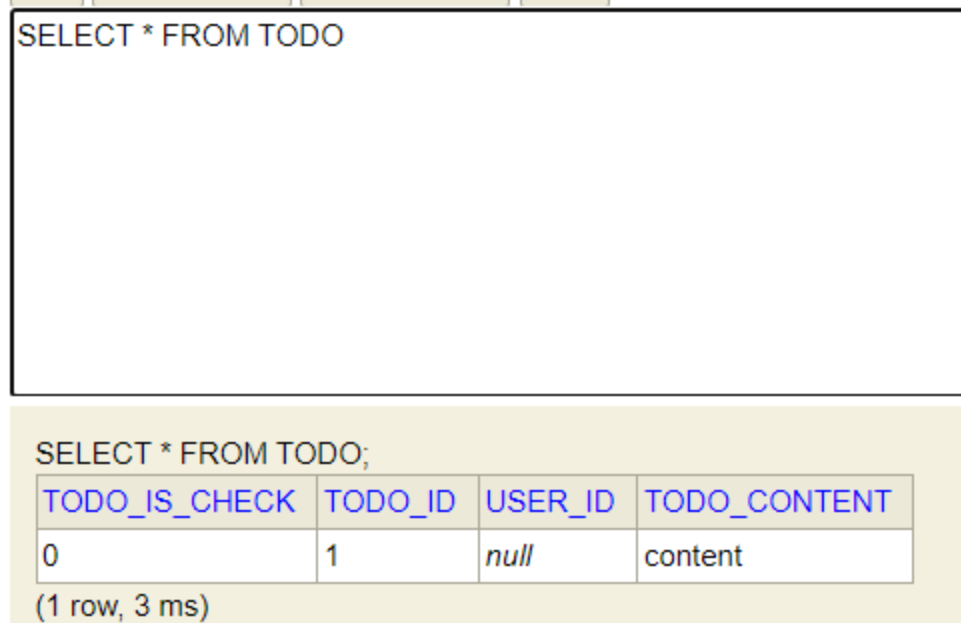
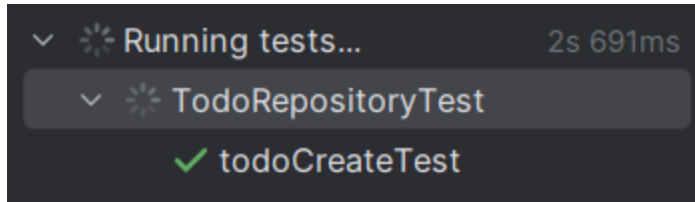
    // when
    todoRepository.save(todo);

    // then
    assertThat(todo.getId()).isNotNull();
}
```

객체가 잘 생성되었다면,
id 값은 자동으로 생성되어
채워져 있을 것이다,

할 일 생성 테스트

- 테스트가 통과해도 미리 작성한 반복문에 의해 프로그램은 종료되지 않는다.
- 콘솔에서 DB에 데이터가 추가된 것을 볼 수 있다.



할 일 조회

- **em.find()**
- 영속성 컨텍스트에서 데이터 조회
영속성 컨텍스트에 데이터가 없다면 DB에서 읽어온다.

```
1 usage new *  
public Todo findById(Long id) {  
    return em.find(Todo.class, id);  
}
```

할 일 조회

- **em.createQuery()**
- 여러 개의 할 일을 조회할 때는 직접 쿼리를 작성한다.
- JPA에서는 DB에서 사용하는 SQL 대신, 객체에 대한 쿼리인 **JPQL**을 사용한다.

```
no usages new *  
public List<Todo> findAll() {  
    return em.createQuery(qlString: "select t from Todo as t", Todo.class)  
        .getResultList();  
}
```


할 일 조회

- **em.createQuery()**
- JPQL은 SQL과 문법이 유사하다.
- 특정 유저의 할 일 리스트를 조회하는 JPQL 은 아래와 같이 where절과 파라미터를 사용하여 작성할 수 있다.

```
no usages new *  
public List<Todo> findAllByUser(User user) {  
    return em.createQuery( queryString "select t from Todo as t where t.user = :user", Todo.class)  
        .setParameter( name: "user", user)  
        .getResultList();  
}
```

SQL vs JPQL

- `select *`
 `from todo`

- todo는 테이블

- `select t`
 `from Todo as t`

- Todo는 엔티티

SQL vs JPQL

- ```
select *
 from todo as t
 where t.user_id
 = @user_id
```

- ```
select t  
  from Todo as t  
  where t.user = :user
```

할 일 조회 테스트 – 단일 데이터

```
@Test
@Transactional
void todoFindOneTest() {
    // given
    Todo todo = new Todo( content: "content", isChecked: false, user: null);
    todoRepository.save(todo);

    // when
    Todo findTodo = todoRepository.findById(todo.getId());

    // then
    assertThat(todo).isNotNull();
    System.out.println(todo.getId());
    System.out.println(todo.getContent());
}
```

할 일 조회 테스트 - 모든 데이터

```
@Test
@Transactional
void todoFindAllTest() {
    // given
    todoRepository.save(new Todo(content: "content1", isChecked: false, user: null));
    todoRepository.save(new Todo(content: "content2", isChecked: false, user: null));
    todoRepository.save(new Todo(content: "content3", isChecked: false, user: null));

    // when
    List<Todo> todos = todoRepository.findAll();

    // then
    assertThat(todos).hasSize(expected: 3);
}
```

할 일 조회 테스트 – 조건

- 유저에 따라 다르게 조회되어야 하므로,
User 레포지토리를 만들고, 유저 생성 기능을 만든다.

할 일 조회 테스트 - 조건

```
@Test
@Transactional
void todoFindAllByUserTest() {
    // given
    User user1 = new User(); User user2 = new User();
    userRepository.save(user1); userRepository.save(user2);

    todoRepository.save(new Todo( content: "content1", isChecked: false, user1));
    todoRepository.save(new Todo( content: "content2", isChecked: false, user1));
    todoRepository.save(new Todo( content: "content3", isChecked: false, user2));

    // when
    List<Todo> todosByUser1 = todoRepository.findAllByUser(user1);
    List<Todo> todosByUser2 = todoRepository.findAllByUser(user2);

    // then
    assertThat(todosByUser1).hasSize( expected: 2);
    assertThat(todosByUser2).hasSize( expected: 1);
}
```

SELECT * FROM TODO;

TODO_IS_CHECK	TODO_ID	USER_ID	TODO_CONTENT
0	1	1	content1
0	2	1	content2
0	3	2	content3

(3 rows, 3 ms)

할 일 수정

- 수정할 할 일을 조회하고 트랜잭션 안에서 객체를 수정하면, 트랜잭션이 끝날 때 엔티티 매니저가 수정 사항을 반영한다.

할 일 수정

- 할 일 content를 업데이트하는 메서드를 엔티티에 추가한다.

```
no usages  new *  
public void updateContent(String content) {  
    |   this.content = content;  
}
```

할 일 수정 테스트

- 데이터의 변경을 DB에서 확인해야 하므로, @Rollback 을 추가

```
@Test
@Transactional
@Rollback(value = false)
void updateTodo() {
    // given
    Todo todo = new Todo( content: "content", isChecked: false, user: null);
    todoRepository.save(todo);
    Todo findTodo = todoRepository.findById(todo.getId());

    // when
    findTodo.updateContent("newContent");
}
```

할 일 수정 테스트

- 데이터의 변경을 DB에서 확인해야 하므로, @Rollback 을 추가

```
Hibernate:
  update
    todo
  set
    todo_content=?,
    todo_is_check=?,
    user_id=?
  where
    todo_id=?
test finish
```

SELECT * FROM TODO;

TODO_IS_CHECK	TODO_ID	USER_ID	TODO_CONTENT
0	1	null	newContent

(1 row, 3 ms)

할 일 삭제

- **em.remove()**
- 영속성 컨텍스트에 엔티티를 올려두고, 해당 데이터를 삭제 상태로 변경한다.

```
no usages  new *  
public void deleteById(Long id) {  
    Todo todo = em.find(Todo.class, id);  
    em.remove(todo);  
}
```

할 일 삭제 테스트

```
@Test
@Transactional
@Rollback(value = false)
void todoRemoveTest() {
    // given
    Todo todo = new Todo( content: "to remove todo", isChecked: false, user: null);
    todoRepository.save(todo);
    todoRepository.save(new Todo( content: "content1", isChecked: false, user: null));
    todoRepository.save(new Todo( content: "content2", isChecked: false, user: null));

    // when
    todoRepository.deleteById(todo.getId());
}
```

할 일 삭제 테스트

```
Hibernate:
  delete
  from
      todo
  where
      todo_id=?
```

SELECT * FROM TODO;

TODO_IS_CHECK	TODO_ID	USER_ID	TODO_CONTENT
0	2	<i>null</i>	content1
0	3	<i>null</i>	content2

(2 rows, 3 ms)

프로젝트 – 과제

- 작성한 모든 엔티티 클래스에 대해 레포지토리 계층을 만들자.

프로젝트 – 과제

- 유저 회원가입을 할 때 CRUD 중 무엇이 필요할까?
- 유저 로그인을 할 때 CRUD 중 무엇이 필요할까?
- 특정 친구의 todo list를 보려면 어떤 JPQL을 작성해야 할까?

프로젝트 – 고민점

- 혹시 과제를 하면서 뭔가 반복적인 작업을 하고 있다는 생각이 들지는 않았는가?

프로젝트 명세

Todo mate API 서버 클론 코딩

<주요 기능>

- 유저 회원가입, 로그인
- 로그인한 유저의 할 일 생성 / 조회 / 수정 / 삭제
- 로그인한 유저의 할 일 체크 / 체크 해제
- 다른 유저에게 친구 요청 / 요청 수락 / 친구 조회 / 친구 삭제
- 친구 유저의 할 일 조회

수고하셨습니다 😊

- 중간고사 모두 화이팅하세요~!